Pearls — Work Order for Antigravity — Thursday, October 29, 2026 (Phase 2)

Prepared by: Janine, at D'Vaughn's direction **For:** Antigravity, on the iMac **Tree:** the thinktank-os tree, Pearls code at `pearls-buildout-2026-10-23/pearls/` **Branch:** `pearls-buildout` (currently at `3cf98d9`, Phase 1). Work, commit, and push on `pearls-buildout` only — never commit directly to `main`. **Status:** This order is self-contained for today. It supersedes the 2026-10-29 Phase 1 order as the operative instruction. Phases 0 and 1 are done and verified (see below) — today's job is Phase 2, Security.

Where things stand (verified, not claimed)

Phase 0 — DONE and independently verified. True Uniswap `v4.0.0` pin (`e50237c`), blob-verified; report corrected; merged as `f46ba91` / `b72f0dc` .

Phase 1 — DONE and independently verified by Janine 2026-10-29 (commit `3cf98d9` , "Phase 1 toolchain setup, v4-periphery v1.0.3 pin, and hook completion verification"):

- Toolchain on the iMac: Foundry found at `~/.config/.foundry/bin/` , symlinked to `~/.foundry/bin` and `~/.local/bin/` ; `forge 1.8.6` ; clean `forge build` .
- `lib/v4-periphery` pinned: tag `v1.0.3` , rev `3779387e5d296f39df543d23524b050f89a62917` in `foundry.lock` . Note for future pin work: Uniswap distributes `v4-periphery` releases via npm — its GitHub git tags are empty, so the tag was established from the npm release line (`@uniswap/v4-periphery@1.0.3`), not from a git tag.
- Interfaces: `PearlHook` implements canonical `IHooks` directly (all 10 callbacks); accounting views and callback signatures aligned to canonical v4 structs.
- CREATE2 salt mining: `HookMiner.sol` mines addresses whose low 14 bits match `PEARL_HOOK_FLAGS = 0x00CC` ; constructor validates via `Hooks.validateHookPermissions` ; unmined addresses revert.
- Fee settlement: `poolManager.take(quoteCurrency, treasury, fee)` inside the unlock context, proven on the Base Sepolia fork in all 4 direction cases, pre- and post-gate, zero below the gate, zero on the crossing swap, quote-orientation invariant.
- Full suite re-run by Janine on a fresh copy of the `3cf98d9` tree: **`forge build` clean, 59 tests passed, 0 failed** (5 suites: 45 unit/fuzz + 14 fork).
- Antigravity-reported, NOT yet independently re-run by Janine (the tools are not on Janine's machine): Halmos 5/5 proofs, Slither 0 High / 0 Medium (13 Low/Informational triaged), token-list generator 3/3, pre-commit language check passed. Phase 2 re-proves the security-tool results with fresh reports — that is part of today's job, not a gap in it.

Carried gap (NOT today's work): `LaunchFactory.POOL_MANAGER` is bound as a constant to the canonical Base Sepolia PoolManager. Multi-network parameterization waits until D'Vaughn schedules it. Do not build it in Phase 2.

Today's tasks — Phase 2: Security, in order

0. Establish the security toolchain where you work. Aderyn, Halmos, and Slither must all run where the work runs (forge already does). Install what's missing on the iMac, record each tool's version in the report, and say exactly where each tool ran if any ran elsewhere. A tool that cannot run is a reported gap, not a skipped step.

1. Aderyn pass on the full core. Run Aderyn over `src/` (core contracts; exclude `test/`, `lib/`, `script/` the way the Slither pass did). Write the full output to `docs/aderyn-<date>.md` and triage EVERY finding into the security report: fixed (with commit reference), accepted with a written reason, or false positive with evidence. Severity counts must be exact. No finding is silently dropped and no finding is fixed by changing a fee, gate, or lock constant.

2. Halmos symbolic verification of the lock and fee invariants. First assess the existing `test/PearlHalmos.t.sol` coverage (5 proofs reported in Phase 1) against each invariant below, then extend proofs wherever coverage is missing. Every invariant ends in one of two states: PROVEN (proof name + run output) or UNPROVEN WITH REASON (what blocks it — that is a finding, not something to hide).

- Lock invariants: Abyssal principal never withdraws; Timed locks release only to the creator and only after unlock; deposit is one-shot; vesting respects the cliff and releases linearly after it.
- Fee invariants: fee is zero below the $10M gate; exactly 30 bps in quote asset only above the gate; the crossing swap pays no fee; the gate is fixed in quote units at deploy; no fee path touches the launched token as the fee asset.
- Structural invariants: no admin key, owner, setter, pause, proxy, or upgrade path exists in core; one canonical pool per launch.

3. Adversarial fork tests against the real PoolManager. Write attacker-perspective tests on the Base Sepolia fork (public `https://sepolia.base.org` default). Each attack must be a named test that demonstrates the attack FAILS (reverts or pays correctly), and each must be listed in the report with what it tried. Minimum coverage — add more where you see an opening:

- Fee bypass: below-gate extraction, crossing-swap double-counting, quote-orientation confusion (quote as currency0 vs currency1), fee-asset substitution.
- Guard windows: max-swap guard enforced then expiring, token-cap bypass attempts, guard expiry edge cases.
- Locker attacks: Abyssal principal withdrawal attempts, timed early withdrawal, one-shot deposit re-entry, compounded-fee collection on dead locks.

- Callback abuse: calls that are not from the PoolManager, reentrancy-shaped paths through the hook callbacks, unmined hook addresses.
- All new tests green. A test that demonstrates a REAL bypass is a critical finding: stop, report it in full, do not quietly patch around it.

4. Publish the full security report. Write `docs/security-report-<date>.md`: scope (contracts + commit hash), tool versions, exact counts (forge per suite, Halmos proofs, Slither severities, Aderyn severities), findings table with disposition for every finding, adversarial tests list, known limitations, and remaining gaps. State explicitly: **no third-party audit exists and none is claimed.** The report is the deliverable — a green run without the report is not done.

5. Commit and push on branch `pearls-buildout`. A fix that is live but uncommitted is not done. Commit and push on the branch where it runs. Do not merge to `main` yourself — that is D'Vaughn's call. Run the pre-commit language check (`bash check-language.sh` from the repo root) before committing — never commit if it fails.

Ground truth (verified, do not re-derive — build on it)

- Uniswap v4-core release tag **`v4.0.0` = commit `e50237c43811bd9b526eff40f26772152a42daba`** (2025-01-21), blob-verified. `SwapParams` / `ModifyLiquidityParams` live in `src/interfaces/IPoolManager.sol`; `src/types/PoolOperation.sol` does not exist in v4.0.0.
- `59d3ecf` is a May 2025 main-branch commit, NOT the v4.0.0 tag. Never repeat the "v4.0.0 @ 59d3ecf" claim anywhere.
- `v4-periphery` is pinned at tag `v1.0.3` / rev `3779387e5d296f39df543d23524b050f89a62917` (npm release line; GitHub git tags for that repo are empty).
- Fee settlement is proven on fork (Phase 1): `poolManager.take(quoteCurrency, treasury, fee)`, all 4 directions, pre/post-gate, zero below gate and on the crossing swap.
- The 2026-10-23 aggregator-hooks prompt remains a SEPARATE pending item awaiting D'Vaughn's paste — it is NOT part of today's tasks unless he says so.

Locked rules — D'Vaughn's decisions, not yours to change

If anything below looks wrong, surface it in your report. Do not remove, rename, relabel, or "fix" it on your own.

- **Fee doctrine, frozen:** the fee is configured at inception per launch, then immutable forever. No platform fee until $10M cumulative volume; above the gate, 0.30% in quote asset only (WETH/USDC); the crossing swap pays no fee; the gate is fixed in quote units at deploy (no swap-time oracle). Tier 1 Patient (locked/burned LP): $10M gate / 0.30%. Tier 3 Uncommitted (no lock): $20K gate / ~0.58% modeled — exact figure unconfirmed, do not hard-code it as locked.

- **No admin keys, no owner, no setters, no pause, no proxy, no upgrade path.** Core contracts have none and gain none.
- **Do not change fee constants, the $10M gate, or lock minimums to make tests pass.** If a design point seems wrong, flag it in the report.
- **Testnet/local only.** Never point anything at mainnet. Deployment anywhere — including Sepolia — is D'Vaughn's explicit call, not part of today's work. `script/Deploy.s.sol` is hard-gated to Base Sepolia (84532) and local Anvil (31337); do not touch that gate.
- **Canonical tree untouched.** `~/workspace/your_files/pearls/` (Janine's canonical copy) stays untouched — sync is D'Vaughn's call.
- **Naming, everywhere including code comments and variable names:** Pearls, always plural. The gold colour and Studio seal are Abyssal Gold. Never use: "sovereign", "ventures", or "institute" for Glenride. User-facing copy never says "AI" — users get help from "the Assistant". (This work order is internal; the rule binds anything user-facing you write.)
- **Honest status, always:** nothing is claimed as live, deployed, audited, or counted until it is. No fabricated testimonials, stats, or endorsements. No third-party audit is claimed until one exists.

Non-goals for today

No deploy. No Sepolia rehearsal launch (that is Phase 3, D'Vaughn's gate — the throwaway test Safe and Pearl #0 are his calls). No grant applications (PRD Phase 2 item 5 applies only once on Base Sepolia testnet — explicitly deferred until Phase 3 exists). No mainnet. No multi-network PoolManager parameterization (carried gap, his schedule). No Phase 4 surfaces, no Plus+/CoinGuard work, no landing-page work, no canonical-tree edits.

Open decisions — D'Vaughn's calls, not yours (do not decide them in code)

Pearl #0 (SPINE manual deploy vs Sepolia dummy) · trading-terminal routing fee · landing-page waitlist/notify path · Plus+ portal auth · canonical-tree sync. Phase 3 (Sepolia + Pearl #0) starts only on his explicit gate.

Report back (required, in BUILD-OUT-REPORT.md + docs/security-report-.md)

- Tool versions + where each tool ran.
- Aderyn: exact severity counts + disposition of every finding.
- Halmos: invariants proven (proof names) + any invariant unproven with reason.
- Adversarial tests: each attack, what it tried, how it failed, test name.
- Full counts: forge per suite, Halmos, Slither, Aderyn, token-list generator, language check.

- Remaining gaps, if any.

Janine independently re-verifies all of it (tool outputs + full suite re-run) before anything is reported to D'Vaughn as sound. Report findings; do not soften them.

Definition of done for today

- Aderyn + Halmos green with every finding documented and dispositioned.
- Adversarial fork tests written and green; any real bypass reported as a critical finding.
- Security report published in `docs/`.
- Report updated, commit pushed on `pearls-buildout`, language check passed.
- Nothing deployed, nothing claimed beyond what the runs prove.